

PAYMENT TRACK, DAY 1: Razorpay keys and test checkout, FRONTEND

Angular 16 + Material, Midnight theme. **1 new service, 2 files replaced, 2 small edits.** Off by default: with `razorpayEnabled: false` the old "Make Payment" flow runs as before and Razorpay's script is never loaded.

Do the **backend guide first** (it explains how to get the keys). If you did this already as "Day 3 afternoon" of the main plan, skip to Day 2.

0. What the user sees

- On the Payment page, with the switch on: a note "Test payment" and the button **Pay with Razorpay**.
- Razorpay's pop-up opens with the key, amount and order id **from the server** (the browser sends no price).
- Close the pop-up: "Payment was cancelled. You were not charged." Bank failure: "Payment failed: ..." and you can try again. A forged signature: the server's message. Success: "Payment was successful!", then **My Enrollments**.
- A course already owned: the server's "You are already enrolled in ..." message and the pop-up never opens (the file includes CHANGES-7's enrolled message, so just replace the two files).

Honest status. Real browser, pretend backend, stand-in Razorpay pop-up: **19 checks with the switch on and 5 with it off, 0 failures**; it also still passes on the merged project that has OTP, instructors and videos. The real Razorpay pop-up and real test cards were not run (they need your keys).

1. Edits

1. `environment.ts` and `environment.prod.ts`

In `angularapp/src/environments/environment.ts` add one line (mind the comma on the line before):

```
export const environment = {
  production: false,
  apiUrl: 'http://localhost:8080',
  aiSearchEnabled: true,
  // ... any other lines you already have (for example googleClientId) stay as they are ...
  // Razorpay test payments: true = the Payment page opens Razorpay Checkout (needs the backend keys).
  false = old behaviour.
  razorpayEnabled: true
};
```

In `environment.prod.ts` add `razorpayEnabled: false` (or `true` when you deploy with keys).

If you leave it `false`, the old "Make Payment" button works exactly as before.

2. `constant.ts`

Inside `export const API = { ... }` add two lines (for example after the `ORDER_BY_CUSTOMER` line):

```
PAYMENT_RAZORPAY_ORDER: '/api/payment/razorpay/order',
PAYMENT_RAZORPAY_VERIFY: '/api/payment/razorpay/verify',
```

3. NEW file `services/payment.service.ts`

`angularapp/src/app/services/payment.service.ts` (new file)

```
import { HttpClient } from '@angular/common/http';
import { Injectable } from '@angular/core';
import { Observable } from 'rxjs';
import { environment } from '../../environments/environment';
import { API, API_URL } from '../constant';

/** What Razorpay Checkout gives the browser after a successful payment. */
export interface RazorpaySuccess {
  razorpay_payment_id: string;
  razorpay_order_id: string;
  razorpay_signature: string;
}

/** Why Checkout ended without a payment. */
export interface RazorpayStop {
  reason: 'dismissed' | 'failed';
  description?: string;
}

/**
 * Razorpay payment for the cart.
 * 1. createRazorpayOrder(): the SERVER prices the cart and creates the Razorpay order.
 * 2. openCheckout(): Razorpay's own pop-up (card, UPI, netbanking...). We never see card details.
 * 3. verifyRazorpayPayment(): the SERVER checks Razorpay's signature and only then saves the order.
 */
@Injectable({
  providedIn: 'root'
})
export class PaymentService {
  private static readonly SCRIPT_URL = 'https://checkout.razorpay.com/v1/checkout.js';
  private apiUrl = API_URL;
  private scriptLoaded: Promise<void> | null = null;

  constructor(private http: HttpClient) {}

  /** True when environment.razorpayEnabled is switched on. */
  get enabled(): boolean {
    return !!environment.razorpayEnabled;
  }

  createRazorpayOrder(): Observable<any> {
    return this.http.post<any>(`${this.apiUrl}${API.PAYMENT_RAZORPAY_ORDER}`, {});
  }

  verifyRazorpayPayment(success: RazorpaySuccess): Observable<any> {
    return this.http.post<any>(`${this.apiUrl}${API.PAYMENT_RAZORPAY_VERIFY}`, {
      razorpayOrderId: success.razorpay_order_id,
      razorpayPaymentId: success.razorpay_payment_id,
      razorpaySignature: success.razorpay_signature
    });
  }

  private loadScript(): Promise<void> {
    if (!this.scriptLoaded) {
      this.scriptLoaded = new Promise<void>((resolve, reject) => {
        const script = document.createElement('script');
        script.src = PaymentService.SCRIPT_URL;
        script.async = true;
      });
    }
  }
}
```

```

        script.onload = () => resolve();
        script.onerror = () => {
            this.scriptLoaded = null;    // allow a retry
            reject(new Error('Could not load Razorpay Checkout'));
        };
        document.head.appendChild(script);
    });
}
return this.scriptLoaded;
}

/** Opens Razorpay Checkout. Resolves with the success data, or rejects with a RazorpayStop. */
async openCheckout(options: any): Promise<RazorpaySuccess> {
    await this.loadScript();
    return new Promise<RazorpaySuccess>((resolve, reject) => {
        const Razorpay = (window as any).Razorpay;
        const checkout = new Razorpay({
            ...options,
            handler: (response: RazorpaySuccess) => resolve(response),
            modal: { ondismiss: () => reject({ reason: 'dismissed' } as RazorpayStop) }
        });
        checkout.on('payment.failed', (response: any) =>
            reject({ reason: 'failed', description: response?.error?.description } as RazorpayStop));
        checkout.open();
    });
}
}

```

4. components/place-order/place-order.component.ts : replace the whole file

angularapp/src/app/components/place-order/place-order.component.ts (replace the whole file)

```

import { Component, NgZone, OnInit } from '@angular/core';
import { FormBuilder, FormGroup, Validators } from '@angular/forms';
import { Router } from '@angular/router';
import { MatDialog } from '@angular/material/dialog';
import { ConfirmDialogComponent } from '../confirm-dialog/confirm-dialog.component';
import { Course } from '../../models/course.model';
import { CartService } from '../../services/cart.service';
import { CustomerService } from '../../services/customer.service';
import { OrderService } from '../../services/order.service';
import { PaymentService, RazorpayStop } from '../../services/payment.service';

@Component({
    selector: 'app-place-order',
    templateUrl: './place-order.component.html',
    styleUrls: ['./place-order.component.css']
})
export class PlaceOrderComponent implements OnInit {
    orderForm: FormGroup;
    courses: Course[] = [];
    errorMessage = '';
    loading = false;
    paymentDone = false;

    constructor(
        private fb: FormBuilder,
        private cartService: CartService,
        private customerService: CustomerService,
        private orderService: OrderService,
        private router: Router,
        private dialog: MatDialog,
        private payment: PaymentService,
        private zone: NgZone
    ) {
        this.orderForm = this.fb.group({
            name: ['', [Validators.required]],
            mobileNumber: ['', [Validators.required, Validators.pattern(/^[0-9]{10}$/)]],

```

```

        information: ['', [Validators.required]],
        email: ['', [Validators.required, Validators.email]],
        totalPrice: [{ value: 0, disabled: true }]
    });
}

get f() { return this.orderForm.controls; }

/** True when the Payment page should use Razorpay (environment.razorpayEnabled). */
get razorpayOn(): boolean { return this.payment.enabled; }

ngOnInit(): void {
    this.cartService.getAllCoursesFromCart().subscribe({
        next: (cart) => {
            this.courses = cart.courses || [];
            this.orderForm.patchValue({ totalPrice: cart.totalAmount || 0 });
        },
        error: () => (this.errorMessage = 'Could not load your cart.')
    });
    this.customerService.viewCustomerByUserId().subscribe({
        next: (customer) => this.orderForm.patchValue({
            name: customer.customerName,
            information: customer.information,
            mobileNumber: customer.user?.mobileNumber,
            email: customer.user?.email
        })
    });
}

makePayment(): void {
    if (this.orderForm.invalid || this.courses.length === 0 || this.loading) { return; }
    // Razorpay switched on and something to pay: the server starts the payment, the order is saved only
    after it is verified
    if (this.razorpayOn && Number(this.orderForm.getRawValue().totalPrice) > 0) {
        this.payWithRazorpay();
        return;
    }
    this.loading = true;
    this.errorMessage = '';
    const order = {
        customer: { customerId: Number(localStorage.getItem('customerId')) },
        courses: this.courses.map(c => ({ courseId: c.courseId })),
        orderPrice: this.orderForm.getRawValue().totalPrice
    };
    this.orderService.addOrder(order).subscribe({
        next: () => {
            // the order is saved, so empty the cart before confirming
            this.cartService.removeAllCourses().subscribe({
                next: () => { this.loading = false; this.showConfirmation(); },
                error: () => { this.loading = false; this.showConfirmation(); }
            });
        },
        error: (err) => {
            this.loading = false;
            // 409 = the server refused because a course in the cart was already bought
            this.errorMessage = err.status === 409
                ? (err.error?.message || 'You are already enrolled in one of these courses.')
                : 'Payment could not be completed. Please try again.';
        }
    });
}

// ----- Razorpay (test mode or live, decided by the keys on the server) -----

private payWithRazorpay(): void {
    this.loading = true;
    this.errorMessage = '';
    this.payment.createRazorpayOrder().subscribe({
        next: (co) => this.openRazorpay(co),
        error: (err) => {
            this.loading = false;

```

```

        this.errorMessage = err.status === 409
        ? (err.error?.message || 'You are already enrolled in one of these courses.')
        : err.status === 400
        ? (err.error?.message || 'Payment could not be started.')
        : 'Payment could not be started. Please try again.';
    }
  });
}

private async openRazorpay(co: any): Promise<void> {
  const form = this.orderForm.getRawValue();
  try {
    const success = await this.payment.openCheckout({
      key: co.keyId,
      amount: co.amount,
      currency: co.currency,
      name: co.name,
      description: co.description,
      order_id: co.razorpayOrderId,
      prefill: { name: form.name, email: form.email, contact: form.mobileNumber },
      theme: { color: '#f59e0b' }
    });
    this.zone.run(() => this.verifyRazorpay(success));
  } catch (e: any) {
    const stop = e as RazorpayStop;
    this.zone.run(() => {
      this.loading = false;
      if (stop?.reason === 'dismissed') {
        this.errorMessage = 'Payment was cancelled. You were not charged.';
      } else if (stop?.reason === 'failed') {
        this.errorMessage = 'Payment failed' + (stop.description ? ': ' + stop.description : '') + '. You
can try again.';
      } else {
        this.errorMessage = 'Razorpay could not be loaded. Check your internet connection and try
again.';
      }
    });
  }
}

private verifyRazorpay(success: any): void {
  this.payment.verifyRazorpayPayment(success).subscribe({
    next: () => {
      // the server saved the order and emptied the cart
      this.loading = false;
      this.showConfirmation();
    },
    error: (err) => {
      this.loading = false;
      this.errorMessage = (err.error?.message || 'We could not confirm your payment.')
        + ' If money was deducted, contact support with payment id ' + success.razorpay_payment_id + '.';
    }
  });
}

private showConfirmation(): void {
  this.paymentDone = true;
  this.dialog.open(ConfirmDialogComponent, {
    disableClose: true,
    data: { title: 'Payment was successful!', message: 'You are now enrolled. Find your courses under My
Enrollments.', confirmText: 'OK', hideCancel: true, icon: 'check_circle' }
  }).afterClosed().subscribe(() => this.closeConfirmation());
}

closeConfirmation(): void {
  this.paymentDone = false;
  this.router.navigate(['/my-orders']);
}
}

```

5. components/place-order/place-order.component.html : replace the whole file

angularapp/src/app/components/place-order/place-order.component.html (replace the whole file)

```
<div class="page">
  <div class="form-card">
    <h2>Payment</h2>
    <p class="lead">Confirm your details to enroll in {{ courses.length }} course{{ courses.length === 1 ?
    '' : 's' }}.</p>
    <p class="form-note" id="razorpayNote" *ngIf="razorpayOn">
      <strong>Test payment.</strong> Razorpay opens in a pop-up. No real money is used in test mode.
    </p>
    <form [formGroup]="orderForm" (ngSubmit)="makePayment()" novalidate>
      <mat-form-field appearance="outline">
        <mat-label>Name</mat-label>
        <input matInput id="name" type="text" formControlName="name" />
        <mat-error *ngIf="f['name'].errors?.['required']">Name is required</mat-error>
      </mat-form-field>
      <mat-form-field appearance="outline">
        <mat-label>Mobile Number</mat-label>
        <input matInput id="mobileNumber" type="text" inputmode="numeric" formControlName="mobileNumber"
        maxlength="10" />
        <mat-error *ngIf="f['mobileNumber'].errors">Enter a valid 10 digit mobile number</mat-error>
      </mat-form-field>
      <mat-form-field appearance="outline">
        <mat-label>What are you (Student / Professional)</mat-label>
        <textarea matInput id="information" rows="2" formControlName="information"></textarea>
        <mat-error *ngIf="f['information'].errors?.['required']">This field is required</mat-error>
      </mat-form-field>
      <mat-form-field appearance="outline">
        <mat-label>Email ID</mat-label>
        <input matInput id="email" type="email" formControlName="email" />
        <mat-error *ngIf="f['email'].errors">Enter a valid email</mat-error>
      </mat-form-field>
      <mat-form-field appearance="outline">
        <mat-label>Total Price</mat-label>
        <span matTextPrefix>&#8377;&nbsp;</span>
        <input matInput id="totalPrice" type="number" formControlName="totalPrice" />
      </mat-form-field>
      <div class="form-error" role="alert" *ngIf="errorMessage"><mat-icon>error_outline</mat-icon>{{
      errorMessage }}</div>
      <div class="form-actions">
        <button mat-flat-button color="primary" type="submit" id="makePayment"
        [disabled]="orderForm.invalid || courses.length === 0 || loading">{{ razorpayOn ? 'Pay with Razorpay' :
        'Make Payment' }}</button>
      </div>
    </form>
  </div>
</div>
```

2. Run it and test it (with the backend keys set)

2a. Full payment in the browser. 1. Set `razorpayEnabled: true` in `environment.ts` and restart `npm start`. 2. Log in as a customer, add a course to the cart, click **Enroll Now**. 3. On the Payment page you see the gold "Test payment" note and the button **Pay with Razorpay**. Click it. 4. Razorpay's pop-up opens. Choose **Card** and use a test card: - Card number **5267 3181 8797 5449** (Mastercard) or **4111 1111 1111 1111** (Visa) - Expiry: any date in the future (for example `12/30`), CVV: any 3 digits (for example `123`), name: anything. - If a card is rejected, open Razorpay's own page **Test card details** (search "Razorpay test card details") and copy a number from there; Razorpay can change them. - Or choose **UPI** and type `success@razorpay` (payment succeeds) or `failure@razorpay` (payment fails). 5. A mock bank

page opens with **Success** and **Failure** buttons. Click **Success**. 6. You should see **"Payment was successful!"**, and the course appears under **My Enrollments**. The cart is empty.

Acceptance checklist

- [] `POST /api/payment/razorpay/order` in Swagger returns `keyId`, an `order_...` id and an amount in paise (keys work)
- [] The Payment page shows "Pay with Razorpay"; a test payment succeeds and the course appears in My Enrollments (and in My Learning when it has videos)
- [] Closing the pop-up or choosing Failure saves no order
- [] A row with status `PAID` exists in `payments`
- [] With `RAZORPAY_ENFORCE=true`, `POST /api/order` answers 403; with `false` it works as before

3. If something goes wrong (frontend)

What you see	Why	Fix
Button still says "Make Payment"	<code>razorpayEnabled</code> is <code>false</code> , or <code>npm start</code> was not restarted	set <code>true</code> , restart, Ctrl+F5
"Razorpay could not be loaded"	no internet, or an ad blocker blocks <code>checkout.razorpay.com</code>	check the network, disable the blocker for localhost
"Payment could not be started"	Razorpay refused the order (wrong key pair)	see the backend guide's table
"Your cart is empty"	nothing in the cart	add a course
"Razorpay needs a total of at least Rs 1"	the total is Rs 0	use a paid course or set <code>razorpayEnabled: false</code>
<code>API.PAYMENT_RAZORPAY_ORDER</code> does not exist	edit 2 missed	add the two lines to <code>constant.ts</code>